

TECNICATURA UNIVERSITARIA EN CIBERSEGURIDAD



**DESARROLLO DE SOFTWARE
SEGURO**

**Trabajo Practico Final
Diseño e Implementación de
APIs Seguras con Flask,
FastAPI y OpenAPI – Tercera
Entrega**

**Cabrera, Ricardo Martin
Deparsia, Ignacio
Quaglia, Tomas
Merlo-San Luis – marzo 2025
Prof: Juan Pablo Villalba
Segundo Año**

Índice

Resumen	4
Introducción	5
Desarrollo	5
1. Análisis Teórico	5
1.1 Estudio detallado de la autenticación por IP	5
1.1.1 Uso de HTTPS	5
1.1.2 Validación de Datos	6
1.1.3 Manejo de Errores	6
1.1.4 Monitoreo	6
1.2 Uso avanzado de OpenAPI	6
1.2.1 Generación automática de documentación y su impacto	6
1.3 Estudio de caso: análisis de una API real desde la perspectiva de seguridad.	7
1.3.1 Twitter API (X API)	7
1.3.2 WhatsApp API	8
1.3.3 Google Maps API	8
2. Implementación Práctica	10
2.1 Tareas:	10
2.1.1 Implementar OAuth 2.0 en el framework no usado en la Entrega 2.	10
2.1.2 Integrar todos los métodos de autenticación en una API unificada.	11
2.1.3 Aplicar medidas de seguridad avanzadas: validación estricta de entradas, simulación de HTTPS y manejo seguro de errores.	3
2.1.4 Generar documentación completa con OpenAPI (usando Swagger UI).	3
2.1.5 Realizar pruebas de seguridad y rendimiento, documentando resultados.	3
Conclusión	8
Bibliografía	9

Resumen

En este trabajo práctico desarrollamos una API segura utilizando Python, con implementaciones tanto en FastAPI como en Flask, abordando distintas técnicas de autenticación y protección de acceso. A lo largo de las tres entregas, fuimos incorporando mecanismos cada vez más avanzados para asegurar las rutas y los datos expuestos por la API.

En las primeras etapas, configuramos rutas que respondían a credenciales válidas y probamos el comportamiento ante credenciales incorrectas. Luego, implementamos API Keys, permitiendo validar el acceso inicial de los usuarios antes de ingresar a las funciones protegidas del sistema.

Más adelante, incorporamos el uso de JWT, lo que nos permitió emitir tokens firmados que representan sesiones de usuario de manera segura. Finalmente, integramos OAuth 2.0 utilizando GitHub como proveedor, lo que nos permitió validar usuarios mediante autenticación externa y obtener un token válido para acceder a recursos autorizados. También terminamos configurando una lógica de bloqueo de IP ante múltiples intentos fallidos, como medida adicional de seguridad.

En conjunto, el trabajo representó un recorrido completo por las distintas formas de proteger una API moderna, entendiendo tanto la lógica técnica como las ventajas de aplicar buenas prácticas desde el diseño. Esta experiencia nos dejó herramientas sólidas para el futuro y una mejor comprensión del rol que juega la seguridad en el desarrollo de software profesional.

Introducción

Esta serie de entregas se centra en el desarrollo de una API segura utilizando herramientas modernas y buenas prácticas en autenticación y autorización. La idea principal es construir una API funcional que, además de ofrecer un "servicio", implemente medidas de seguridad fundamentales que protejan tanto la integridad de los datos como el acceso a los mismos.

Durante el desarrollo abordaremos distintos métodos de autenticación, comenzando con mecanismos más simples como el uso de credenciales, para luego avanzar hacia soluciones más robustas como la implementación de tokens JWT, el uso de claves de acceso (API Keys), y la autenticación delegada a través de OAuth 2.0. Además, se buscará incorporar ciertas protecciones complementarias, como el bloqueo por intentos fallidos, y se trabajará con ejemplos prácticos para probar el comportamiento del sistema ante distintos escenarios.

Este informe tiene como objetivo no solo mostrar el funcionamiento de cada uno de estos mecanismos, sino también reflexionar sobre su importancia en el desarrollo de aplicaciones modernas y la necesidad de pensar en seguridad desde las primeras etapas del diseño de un software.

Desarrollo

1. Análisis Teórico

1.1 Estudio detallado de la autenticación por IP

En el desarrollo y mantenimiento de APIs seguras, existen prácticas fundamentales que deben adoptarse para proteger la integridad, confidencialidad y disponibilidad de los datos. A continuación, se detallan las mejores prácticas para garantizar la seguridad de las APIs, basadas en estándares de la industria como OWASP y otras fuentes confiables.

1.1.1 Uso de HTTPS

El uso de **HTTPS** es una de las medidas más básicas pero esenciales, ya que garantiza que la comunicación entre el cliente y el servidor esté cifrada mediante TLS. Esto previene ataques como el *man-in-the-middle* y protege la información sensible, especialmente en entornos donde se transmiten tokens de autenticación, credenciales u otros datos críticos.

1.1.2 Validación de Datos

La **validación estricta de datos** es otra práctica clave, tanto del lado del cliente como del servidor. Las APIs deben validar todos los datos de entrada (por ejemplo, parámetros de consulta, encabezados, cuerpos JSON) para prevenir ataques como inyecciones SQL, XSS, comandos maliciosos o datos inesperados que puedan comprometer la lógica de negocio. Esta validación debe ser basada en listas blancas, limitando los datos a formatos y valores aceptables, como correos electrónicos o números de teléfono, y sanitizar entradas para eliminar caracteres maliciosos, en lugar de depender de listas negras. Por ejemplo, una API que espera un ID numérico debe rechazar cualquier entrada no numérica

1.1.3 Manejo de Errores

El **manejo adecuado de errores** también forma parte de una arquitectura segura. Las respuestas de error deben estar diseñadas para no revelar información sensible como trazas de pila, detalles del backend o configuraciones del servidor. Idealmente, los errores deben ser genéricos hacia el cliente sin proporcionar información útil a los atacantes, pero detallados en los registros internos para su posterior análisis y resolución por parte de los desarrolladores. Las APIs deben devolver códigos de estado HTTP estandarizados (ej., 400 para solicitudes inválidas, 401 para autenticación fallida) y mensajes descriptivos sin exponer detalles sensibles.

1.1.4 Monitoreo

El **monitoreo y registro continuo** (logging) permite detectar accesos inusuales, intentos de explotación, picos de solicitudes o fallas en la API. Las herramientas de monitoreo permiten analizar patrones de uso, identificar brechas y aplicar reglas de seguridad proactivas. Herramientas como Prometheus o Datadog ofrecen análisis en tiempo real, mientras que el registro detallado facilita auditorías forenses. La integración de soluciones SIEM (Security Information and Event Management) puede ayudar a correlacionar eventos y responder ante incidentes en tiempo real. Implementar la limitación de tasas (rate limiting) ayudaría proteger contra ataques DDoS, restringiendo el número de solicitudes por usuario

1.2 Uso avanzado de OpenAPI

OpenAPI Specification (OAS) viene revolucionado, en estos últimos años, la forma en que los desarrolladores diseñan, consumen y aseguran APIs.

1.2.1 Generación automática de documentación y su impacto

Uno de sus usos más poderosos es la **generación automática de documentación**, que permite definir todos los endpoints, parámetros, métodos HTTP, tipos de datos y respuestas esperadas en un archivo estructurado (JSON o YAML). Esta especificación

puede ser interpretada por herramientas como Swagger UI o Redoc, generando documentación legible y navegable tanto para desarrolladores como para auditores de seguridad. Herramientas como Swagger UI también permiten generar interfaces interactivas que facilitan a los desarrolladores explorar endpoints, parámetros y respuestas esperadas.

Desde una perspectiva de ciberseguridad, esta documentación detallada tiene un **doble impacto**. Por un lado, mejora la transparencia, reduce los errores de implementación y facilita la integración segura de clientes, ya que los consumidores saben exactamente qué esperar de la API. Por otro lado, si no se gestiona correctamente (por ejemplo, dejando expuesta una documentación en producción sin autenticación), puede aumentar la superficie de ataque, brindando a un atacante potencial una "hoja de ruta" para explorar y explotar endpoints mal asegurados. Plataformas como Postman o Apidog integran OpenAPI para validar solicitudes y respuestas, asegurando que cumplan con las especificaciones definidas. Además, OpenAPI soporta la definición de esquemas de seguridad (ej., OAuth 2.0, JWT), lo que refuerza la autenticación y autorización.

El uso avanzado de OpenAPI también permite la validación automática de esquemas, la generación de pruebas automatizadas (como análisis estático (SAST) y dinámico (DAST)) y la verificación de cumplimiento con requisitos de seguridad (por ejemplo, tokens requeridos, cifrado, políticas de CORS). De esta forma, OpenAPI no solo documenta, sino que **fortalece la seguridad desde el diseño**, apoyando los principios de *Security by Design* y *Shift Left* en el ciclo de desarrollo seguro (SSDLC).

1.3 Estudio de caso: análisis de una API real desde la perspectiva de seguridad.

A continuación, se analizan las APIs de Twitter (ahora X API) y WhatsApp desde la perspectiva de ciberseguridad, evaluando sus medidas de seguridad y posibles riesgos.

1.3.1 Twitter API (X API)

Esta API permite acceder a múltiples funcionalidades como publicaciones, interacciones, análisis de métricas, entre otros, y está disponible tanto para uso público como empresarial. Twitter ha implementado varias medidas de seguridad en su API, comenzando por el uso obligatorio de **HTTPS** para todas las comunicaciones, lo cual protege los datos en tránsito.

En cuanto a **autenticación**, la API utiliza mecanismos robustos como OAuth 2.0, permitiendo a las aplicaciones obtener tokens de acceso con diferentes niveles de permisos. Este enfoque es fundamental para aplicar el principio de **menor privilegio**, limitando el acceso a solo los recursos necesarios. Además, Twitter aplica **rate limiting**, que impide que una misma aplicación realice demasiadas solicitudes en un corto período de tiempo, reduciendo la efectividad de ataques de denegación de servicio (DoS) o fuerza bruta.

En cuanto a la exposición de errores, Twitter ha estructurado sus respuestas para evitar fugas de información sensible. Sin embargo, como toda API pública, presenta riesgos asociados al uso indebido por parte de terceros, como el **scraping** masivo o la **automatización de bots**, lo que ha generado desafíos incluso políticos y éticos para la plataforma. Para mitigar estos riesgos, Twitter implementa una política de revisión y aprobación de aplicaciones que deseen acceder a su API, junto con monitoreo y suspensión de aquellas que violan los términos de uso.

1.3.2 WhatsApp API

La API de WhatsApp Business, ampliamente utilizada por empresas para automatizar comunicaciones con clientes. Esta API permite enviar y recibir mensajes mediante solicitudes HTTP autenticadas y se integra con plataformas como Meta Business Manager.

Uno de los aspectos destacables en términos de seguridad es su uso obligatorio de HTTPS y OAuth 2.0 para la autenticación, lo que garantiza comunicaciones cifradas y control de acceso mediante tokens de corta duración.

Además, WhatsApp impone límites de tasa (rate limiting) para evitar abusos del servicio, como el envío masivo de mensajes o ataques de denegación de servicio (DoS).

La API también ofrece validación estricta de los datos que se envían, incluyendo validaciones de tipo, estructura y formato, lo que reduce el riesgo de inyecciones o malformaciones de mensajes.

A nivel de monitoreo, se integran métricas y registros detallados accesibles desde el portal de Meta, lo que permite auditar eventos y realizar análisis forenses en caso de incidentes. Sin embargo, como toda API expuesta públicamente, no está exenta de riesgos, especialmente si se expone sin el control adecuado de tokens o si los endpoints se filtran o quedan accesibles sin protección.

Por ello, es vital que las empresas que la integran implementen controles adicionales como listas blancas de IP, rotación de claves y monitoreo en tiempo real.

1.3.3 Google Maps API

Es una de las APIs más utilizadas a nivel mundial para la geolocalización y servicios de mapas, presente en miles de aplicaciones. A nivel de seguridad, esta API implementa múltiples capas de protección, pero también presenta ciertos desafíos.

En primer lugar, Google Maps API requiere un **API Key** para su utilización. Este token funciona como identificador y mecanismo de control de acceso, aunque no debe considerarse como una medida de seguridad suficiente. Por ello, Google recomienda **restringir el uso del API Key** mediante la configuración de restricciones por dominio (referer), por IP, o por aplicación (paquete Android o iOS). No seguir estas recomendaciones puede permitir que un atacante reutilice el API Key y genere costos elevados al propietario legítimo.

En cuanto a las comunicaciones, todo el tráfico entre el cliente y los servidores de Google se realiza mediante **HTTPS**, garantizando la confidencialidad de los datos transmitidos. No obstante, si se utiliza la API desde el lado del cliente (JavaScript), aún puede haber riesgo de **exposición del API Key** en el código fuente o en herramientas de inspección del navegador.

Además, la API impone **límites de uso (rate limiting)**, autenticación adicional para APIs más sensibles (como Places API), y soporte para OAuth 2.0 en integraciones empresariales. A pesar de estas protecciones, el verdadero reto para los desarrolladores es implementar una **seguridad complementaria del lado servidor**, como ocultar claves sensibles y restringir el acceso a la lógica crítica mediante un backend.

También es destacable que Google ofrece **logs y métricas detalladas** sobre el uso de su API, lo cual permite monitorear patrones inusuales y aplicar medidas de prevención. Este enfoque de seguridad integral, donde se combinan controles técnicos, configuraciones adecuadas y monitoreo constante, sirve como ejemplo a seguir en el diseño de APIs modernas.

1.4 Reflexión crítica sobre Flask vs. FastAPI tras su implementación práctica.

Como primera impresión, ambos frameworks resultaron muy similares y estructurados. Además, al compartir Python como idioma de programación base, las similitudes se destacan aún más. También el método de instalar complementos puntuales de desarrollo con librerías estándares disponibles para diversos objetivos puntuales facilitan las implementaciones.

FastApi resulto más robusta para las implementaciones ya que por ejemplo incluye de manera directa Swagger para generar documentación, mientras que Flask necesita agregarlo como librería.

Flask si bien es mucho más liviana y modesta, no se priva de tener gran potencial operativo. Ambos FrameWorks son ideales para las implementaciones requeridas. A nuestro criterio Flask nos resultó más configurable y adaptable, por lo cual la elegimos como FrameWork para la implementación final.

2. Implementación Práctica

2.1 Tareas:

2.1.1 Implementar OAuth 2.0 en el framework no usado en la Entrega 2.

Se ha implementado la autenticación de OAuth 2.0 con los servicios de GitHub. Para esto ya debemos tener una cuenta registrada con anterioridad en GitHub, esta cuenta será la base para configurar los servicios de autenticación que implementaremos posteriormente. Así pues accedemos a GitHub y nos dirigimos a la sección de desarrolladores, en <https://www.github.com/settings/developers> y creamos una nueva aplicación OAuth.

Debemos completar los datos requeridos, como ser *Nombre de la aplicación*, que en nuestro caso es **API UGR Protegida**, la dirección de la Home Page, que para nosotros es <http://127.0.0.1:5000>, y la dirección de autorización de retorno, o Authorization callback URL, que en nuestro caso es <http://127.0.0.1:5000/callback>. Es muy importante que estas dos direcciones sean exactamente las mismas que se usaran en la implementación ya que en caso contrario, no funcionara la autenticación.

Application logo



Upload new logo

You can also drag and drop a picture from your computer.

Badge background color

#ffffaa

The hex value of the badge background color.



Application name *

API UGR Protegida

Something users will recognize and trust.

Homepage URL *

<http://127.0.0.1:5000>

The full URL to your application homepage.

Application description

API UGR Protegida

This is displayed to all users of your application.

Authorization callback URL *

<http://127.0.0.1:5000/callback>

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Elaboracion propia

Ya con esta configuración activa y con la ayuda de Chat GPT, modificamos el código anterior para que sea incluida la validación por GitHub. Los datos de configuración

como CLIENT ID y CLIENT SECRET son adosados a nuestro archivo de variables de entorno.

API UGR Protegida

 Rimaca1973 owns this application. [Transfer ownership](#)

You can list your application in the [GitHub Marketplace](#) so that other users can discover it. [List this application in the Marketplace](#)

1 user [Revoke all user tokens](#)

Client ID
Ov231ikR0NREEjAm2yIR

Client secrets [Generate a new client secret](#)

 *****bb9a10d6
Added 4 days ago by Rimaca1973
Last used within the last week
[Delete](#)
You cannot delete the only client secret. Generate a new client secret first.

Elaboracion propia

Los mismos serán cotejados en el circuito de verificación.

Nuestro archivo .env queda entonces de la siguiente manera:

```
API_KEY=supersecreta123
JWT_SECRET=secretojwt123
ALLOWED_IPS=192.168.1.10,192.168.1.98,127.0.0.1
ADMIN_USER=admin
ADMIN_PASSWORD=secret
GITHUB_CLIENT_ID=Ov231ikR0NREEjAm2yIR
GITHUB_CLIENT_SECRET=25d1163e575b127a3f4feb52c21936cdbb9a10d6
OAUTHLIB_INSECURE_TRANSPORT=1 # esto permite usar HTTP localmente
```

Elaboracion propia

Hemos tomado la ayuda de ChatGPT para realizar el circuito completo de verificación con GitHub. Esto lo mostraremos en la siguiente sección del trabajo.

2.1.2 Integrar todos los métodos de autenticación en una API unificada.

Hemos desarrollado el código implementado las autenticaciones requeridas de a una, para verificar el correcto funcionamiento de las mismas de manera directamente práctica, nuestra implementación final, verifica:

- APIKEY
- ACCESO IP
- Credenciales de Usuario y contraseña
- Verificacion OAuth con servicios proporcionados por GitHub.

Esta parte la mostraremos de manera operativa en la sección de pruebas de seguridad y rendimientos.

2.1.3 Aplicar medidas de seguridad avanzadas: validación estricta de entradas, simulación de HTTPS y manejo seguro de errores.

Dentro de las medidas de seguridad avanzadas, hemos implementado una protección contra ataques de fuerza bruta, que bloquea la IP después de tres intentos fallidos. también la validación estricta de entradas validando la IP de acceso, la inclusión de la validación también estricta de la ApiKey.

Si bien no está implementado por ser un ejemplo práctico en entorno local, el código es totalmente compatible con la implementación de servicios HTTPS, obligatorio para los entornos de trabajo actuales.

Podríamos ampliar aún más, validando longitud y tipos de caracteres, limitando el tamaño de los payloads o implementar un rate limiting entre otras opciones, pero consideramos que se escapan a la idea de este trabajo práctico, al cual las implementaciones aplicadas lo hacen sólido y robusto, aunque también recordamos que nunca es suficiente la seguridad implementada.

2.1.4 Generar documentación completa con OpenAPI (usando Swagger UI).

Hemos recopilado la información generada por Swagger en el archivo [tp003-flask-swagger.pdf](#) que se encuentra disponible en el repositorio de GitHub.

2.1.5 Realizar pruebas de seguridad y rendimiento, documentando resultados.

Realizamos las pruebas de seguridad de manera empírica sobre el servidor que monta nuestro código.

La primera revisión la hacemos con una APIKEY inválida. Esta APIKEY la ingresamos vía el request en el navegador. Aclaramos también que, en este proceso, se piden también las credenciales de usuario y contraseña.

Así pues, ingresamos: <http://127.0.0.1:5000/protégida/?apikey>

Donde no especificamos la APIKEY o ingresamos una APIKEY errónea como ser : <http://127.0.0.1:5000/protégida/?apikey=supersecreta> En ambos casos tendremos el aviso de:



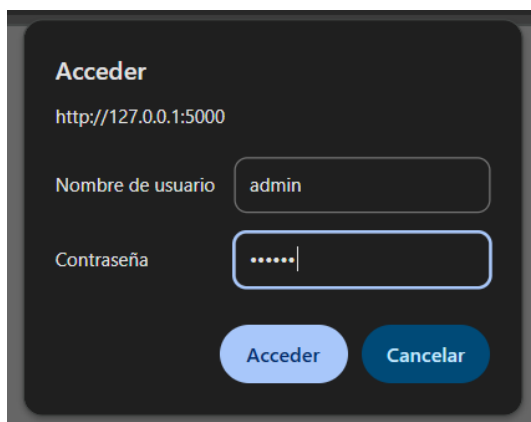
API Key inválida

Elaboración propia

En el caso de ingresar la APIKEY correcta:

<http://127.0.0.1:5000/protegida/?apikey=supersecreta123>

Se nos pedirán las credenciales correspondientes, allí en primer lugar indicamos las credenciales correctas, admin y secret:



Acceder

http://127.0.0.1:5000

Nombre de usuario admin

Contraseña

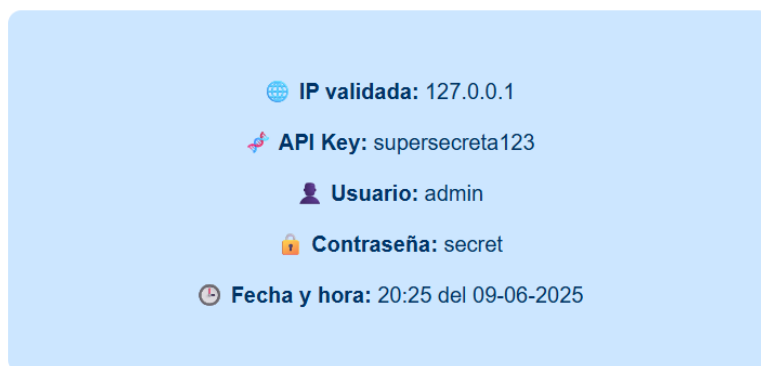
Acceder Cancelar

Elaboracion propia

El resultado será una pantalla informativa:



Verificación completada



 **IP validada:** 127.0.0.1

 **API Key:** supersecreta123

 **Usuario:** admin

 **Contraseña:** secret

 **Fecha y hora:** 20:25 del 09-06-2025

 Autenticación OAuth GitHub

Elaboracion propia

Recalcamos como siempre que estos datos son solo a titulo informativo y nos muestran la colección de credenciales que se van recolectando.

Al hacer click en el botón de Autenticación OAuth GitHub nos llevara a la siguiente pantalla:





Sign in to **GitHub**
to continue to **API UGR Protegida**

Username or email address

Password

[Forgot password?](#)

Sign in

[Sign in with a passkey](#)

New to GitHub? [Create an account](#)

Elaboracion propia

Aquí ya pasamos el control de la verificación de usuario al sistema de GitHub. Es necesario tener una cuenta valida ya registrada en GitHub para poder continuar, en caso de ingresar datos no registrador, obtendremos la siguiente respuesta :



Incorrect username or password. 

Elaboracion propia

Y en caso de no ingresar datos validados por GitHub, nuestro flujo de información se detiene aquí.

En caso de ingresar credenciales correctas, estas serán cotejadas con las almacenadas por GitHub, que, en caso de ser validadas de manera correcta, veremos la siguiente página:



✓ Autenticación OAuth Exitosa

Bienvenido, Rimaca1973!

gho_foeTK3xn670AZ6apW8QUv30PqjHtqL2HSHVQ

 **Ingresar a Ruta Protegida**

Elaboracion propia

Y luego, al ir al botón de Ingresar a Ruta Protegida



Acceso Concedido

 **Usuario:** admin

 **Contraseña:** secret

 **API Key:** supersecreta123

 **IP origen:** 127.0.0.1

✓ **IPs permitidas:** 192.168.1.10, 192.168.1.98, 127.0.0.1

 **Ingreso a las:** 20:32 del 09-06-2025

GitHub: Rimaca1973

gho_foeTK3xn670AZ6apW8QUv30PqjHtqL2HSHVQ

 Datos de ejemplo con fines académicos

Elaboracion propia

Tendremos la parte final de la validación de todas las credenciales y métodos de control implementados.

Quedo por mostrar la validación de credenciales, que en este caso hemos limitado a tres intentos incorrectos. Luego de lo cual, se bloqueará el acceso a la IP.



✖ Acceso bloqueado temporalmente

Demasiados intentos fallidos desde IP: 127.0.0.1

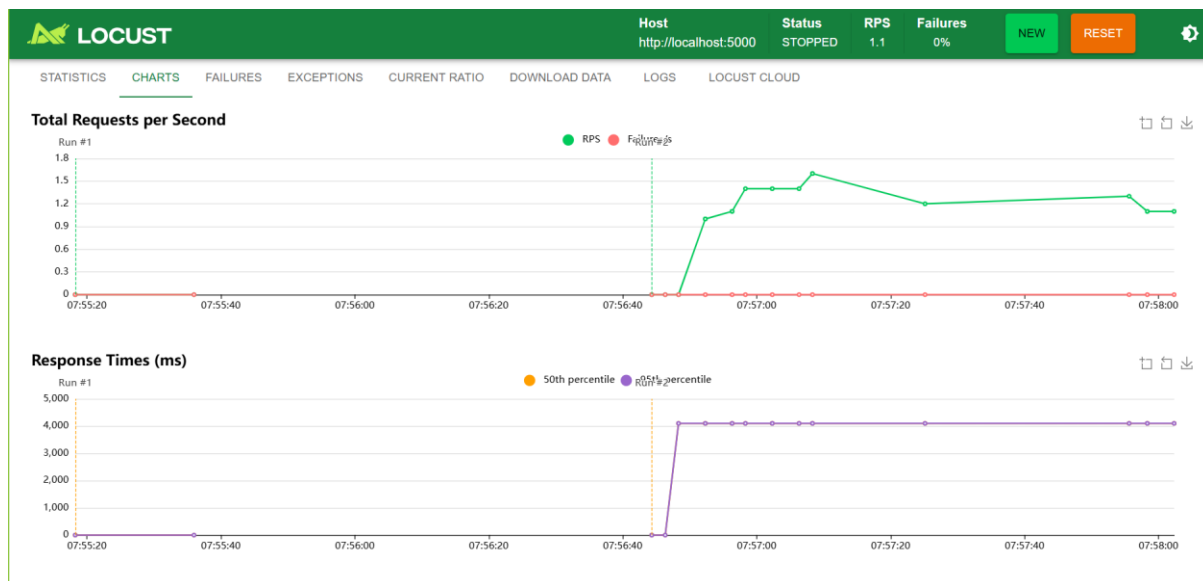
Elaboracion propia

mismo se encuentran en el archivo Locust.pdf.

Aquí mostramos algunas imágenes de los benchmark de rendimiento y detalles informativos

LOCUST												Host	Status	RPS	Failures	NEW	RESET	⚙
												http://localhost:5000	STOPPED	1.1	0%			
STATISTICS																		
CHARTS																		
FAILURES																		
EXCEPTIONS																		
CURRENT RATIO																		
DOWNLOAD DATA																		
LOGS																		
LOCUST CLOUD																		
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s						
GET	/protegida/	107	0	4100	4100	4100	4084.17	4040	4130	967	1.1	0						
	Aggregated	107	0	4100	4100	4100	4084.17	4040	4130	967	1.1	0						

Elaboracion propia



Elaboracion propia

Por último, también se encuentra disponible en el repositorio el video demostrando el flujo de trabajo de la API, el mismo es www.rimaca.com.ar/tp003.mp4

Adicionalmente el código final se encuentra en el archivo [tp003.py](#). Y también hemos actualizado los archivos [requirements.txt](#) y [.env](#)

Conclusión

A lo largo de las tres entregas que conformaron este trabajo práctico, fuimos desarrollando progresivamente una API segura, implementando diferentes mecanismos de autenticación como API Key, JWT y OAuth 2.0. Desde una primera instancia más exploratoria y conceptual, hasta llegar a un producto funcional y robusto, el proceso nos permitió no solo adquirir conocimientos técnicos fundamentales en seguridad de software, sino también enfrentarnos a los desafíos reales que implica diseñar y proteger una API moderna.

Durante la primera entrega, nos familiarizamos con los conceptos clave de autenticación y autorización, y analizamos distintos enfoques en cuanto a diseño seguro. En esta etapa se destacaron los primeros pasos, aunque con ciertas debilidades propias del inicio, como la falta de estructura y una implementación algo desorganizada. Sin embargo, esto fue una gran base para nuestras siguientes entregas nos.

En la segunda etapa, dimos un salto significativo incorporando prácticas de autenticación mediante API Key, protección por IP y JWT, utilizando principalmente FastAPI. Aquí, la implementación se volvió más técnica y completa: se configuró acceso restringido a determinadas IPs, se gestionaron tokens JWT con control de acceso, y se documentó cada paso con pruebas y resultados. A su vez, se evaluaron las ventajas de FastAPI frente a Flask, optando por el primero debido a su claridad, velocidad y facilidad para integrar herramientas modernas.

Ya en la tercera y última entrega, integramos todos los conocimientos anteriores en una API completa. Se logró una autenticación exitosa por API Key, en donde el sistema reconocía claves válidas e inválidas, con acceso progresivo según las credenciales ingresadas. Además, se incorporó OAuth 2.0 mediante GitHub, permitiendo validar usuarios a través de una plataforma externa y recibir un token de acceso seguro. Un detalle interesante fue la implementación de un sistema de bloqueo automático de IPs tras varios intentos fallidos de autenticación, fortaleciendo aún más la seguridad del sistema.

Este proceso hubo de todo, desde errores de conexión y validación, hasta desafíos con los tokens o restricciones por IP, cada obstáculo nos sirvió para seguir aprendiendo. Sin embargo, lo más destacable fue cómo fuimos capaces de construir un sistema complejo paso a paso, de forma clara, ordenada y funcional.

Podemos afirmar que este trabajo no solo fue una buena experiencia formativa, sino también una interesante práctica profesional. Nos permitió aplicar buenas prácticas de seguridad en el desarrollo de APIs, comprender riesgos comunes y cómo mitigarlos, y ver de manera concreta cómo la correcta implementación de mecanismos de autenticación puede mejorar la experiencia de usuarios, empresas y desarrolladores. Concluimos con la certeza de haber dado un gran paso en nuestro aprendizaje como profesionales de la ciberseguridad y el desarrollo seguro.

Bibliografía

- Python Software Foundation. (s.f.). base64 – Base16, Base32, Base64, Base85 Data Encodings. Recuperado de <https://docs.python.org/3/library/base64.html>
- Parasoft. (2022). Guía de pruebas de seguridad API. <https://es.parasoft.com>
- Paradigma. (2019). Seguridad en tus APIs: cómo evitar ataques de securización. <https://www.paradigmadigital.com>
- Montero, G. (2025). Las mejores prácticas API que todo desarrollador debe conocer. <https://salyseo.com>
- Prometeo. (2024). Seguridad en APIs: Mejores prácticas y riesgos comunes. <https://prometeoapi.com>
- Apidog. (2025). Las 10 mejores prácticas de la API REST para desarrolladores. <https://apidog.com>
- Twitter. (2023). Twitter API documentation. <https://developer.twitter.com/en/docs/twitter-api>
- WhatsApp. (2023). WhatsApp Business API documentation. <https://developers.facebook.com/docs/whatsapp>
- Meta for Developers. (2024). *WhatsApp Business API Documentation*. Meta. <https://developers.facebook.com/docs/whatsapp>
- OpenAI. (2025). ChatGPT (versión GPT-4) [Modelo de lenguaje]. <https://chat.openai.com/>
- xAI. (2025). Grok 3 (Versión de software de inteligencia artificial). xAI. <https://x.ai/grok>